

Naoyun SDK Integration Guide

Table of Contents

Note on Use Cases	3
Purchasing SDK	3
Frequently Asked Questions	3
Sample Description	4
C# SDK Main Components	4
Initialize	4
SDK Version	5
Set Callback Functions	5
BLE Scanning	5
Stop Scanning	6
Connect Device	6
Disconnect	6
Server Authentication	7
Device Status	7
Noise Reduction Mode	8
Touch Control	8
Smart Playback	8
Start Data Acquisition	9
Stop Data Acquisition	9

Raw EEG Data9

Get Latest 1–60 Seconds of EEG Data 10

Signal Quality & Spectrum Data 10

Real-Time Mental State Feedback 11

Save CSV Example 12

Real-Time Waveform Example 12

Real-Time Spectrum Example 12

Note on Use Cases

Because this product utilizes BLE data transmission, it is not suitable for ERP (Event-Related Potential) experiments that require high real-time synchronization.

Purchasing SDK

Get appID and Secret, contact: contact@brainrhythm.cn or support@veetra.ai.

Download samples: <https://github.com/brainrhythm>, currently available in Python and C# versions.

Python SDK version download link: https://github.com/brainrhythm/Naoyun_SDK_Python

C# SDK version download link: https://github.com/brainrhythm/Naoyun_SDK_Csharp

Verify the order, and we will send the appID and Secret via email.

Frequently Asked Questions

If the device cannot be found during scanning, troubleshoot as follows:

- Make sure the headset is turned on, or put it back into the charging case and take it out 3 seconds after the light turns on;
- Check whether the PC/laptop's Bluetooth supports BLE functionality;
- Check whether the headset's BLE is already connected to another device; kill the Naoyun/Veetra app on the phone from the background and then search again.

For the Python version, ensure that the library files and `naoyundemo.py` are in the same directory. Run the command in the directory where `naoyundemo.py` is located,

- for example: `python naoyundemo.py ak_XXXXXX sk_XXXXXX`

Sample Description

Recommended minimum version: Visual Studio 2022 with .NET Framework 4.7.2

C# SDK Main Components

- `NaoyunSdk.dll` — Encapsulates Bluetooth protocols and server authentication protocols
 - `libmath.dll` — Encapsulates the algorithm library
 - `INaoyunSdkApi.cs` — Public API interface class
 - `SdkService.cs` — SDK usage reference service example
 - `NaoyunSdkSample.csproj` — Reference project file example
 - `MainForm.cs` — Main interface including BLE scanning, connection, and other control examples
 - `EegWaveformForm.cs` — Real-time EEG waveform and spectrum chart examples
-

Initialize

In `SdkService.cs`, modify your `accessKeyId` and `accessKeySecret`:

```
private string accessKeyId = "your accessKeyId";  
private string accessKeySecret = "your accessKeySecret";
```

```
_sdkApi = new NaoyunSdkApi();  
_sdkApi.Initialize(id, secret, isDomestic);
```

This initialization passes the `id` and `secret` to the SDK. The SDK will interact with the server during BLE connection to obtain authorization information.

- `isDomestic`: Set to `true` for users in mainland China; set to `false` for others.

SDK Version

```
sdkVersion = _sdkApi.GetSdkVersion();
```

Returns the SDK version string.

Set Callback Functions

```
_sdkApi.BleDeviceDiscovered += SdkApi_BleDeviceDiscovered;           // Scan callback
_sdkApi.ConnectionStateChanged += SdkApi_ConnectionStateChanged;      // Connection
state callback
_sdkApi.DeviceStatusNotificationReceived += SdkApi_DeviceStatusNotificationReceived; //
Device status callback
_sdkApi.DataReceived += SdkApi_DataReceived;                          // Raw data
callback
_sdkApi.SpectrumDataReceived += SdkApi_SpectrumDataReceived;          // Signal
quality & spectrum callback
_sdkApi.SpectrumTaskStatusChanged += SdkApi_SpectrumTaskStatusChanged; // Spectrum
task status callback
_sdkApi.ServerAuthCompleted += SdkApi_ServerAuthCompleted;           //
Authentication callback
_sdkApi.MentalStateDataReceived += SdkApi_MentalStateDataReceived;    // Mental state
callback
```

BLE Scanning

```
await _sdkApi.StartBleScanAsync(TimeSpan.FromSeconds(secondTime));
```

Parameter: Scan timeout duration, in seconds.

Callback:

```
private void SdkApi_BleDeviceDiscovered(object sender, BleDeviceInfo device)
```

Discovered BLE devices will be reported via this callback.

Structure:

```
public class BleDeviceInfo
{
    public string Id { get; set; }
    public string Name { get; set; }
    public ulong BluetoothAddress { get; set; }
    public bool IsConnected { get; set; }
};
```

Stop Scanning

```
_sdkApi.StopBleScanAsync();
```

Connect Device

```
bool success = await _sdkApi.ConnectBleAsync(_selectedDevice.BluetoothAddress);
```

Select one of the discovered devices to connect.

Callback:

```
private void SdkApi_ConnectionStateChanged(object sender, DeviceStateEventArgs e)
```

Feedback on current connection state when BLE device connects or disconnects:

- Connected: **Connected to {device.name}**
- Disconnected: **Disconnected from {device.name}**

Disconnect

```
await _sdkApi.DisconnectAsync();
```

Server Authentication

```
private void SdkApi_ServerAuthCompleted(object sender, ServerAuthResultEventArgs e)
```

During BLE connection, the SDK automatically retrieves authorization information from the server.

Each AppID and Secret pair supports binding up to 5 devices by default. Devices are automatically bound upon connection.

- Authentication succeeded
- Authentication failed

Device Status

```
private void SdkApi_DeviceStatusNotificationReceived(object sender,  
DeviceStatusNotificationEventArgs e)
```

The callback reports the latest status when connection succeeds or status changes:

- Hardware/Software version
- Left/Right ear battery level (0–100%)
- Left/Right ear wearing status
- Noise reduction mode (Normal, Noise Reduction, Ambient Sound)
- Touch control switch
- Smart playback (when enabled, playback pauses when earbuds are removed and resumes when worn again)

Structure:

```
public class DeviceStatusNotificationEventArgs : EventArgs  
{  
    internal ushort Command { get; private set; }  
    public byte EarSide { get; private set; }  
    public byte LeftBattery { get; private set; }  
    public byte RightBattery { get; private set; }  
    public bool LeftWorn { get; private set; }  
    public bool RightWorn { get; private set; }  
}
```

```
public byte HardwareVersion { get; private set; }
public byte SoftwareVersion { get; private set; }
internal bool IsBigEndian { get; private set; }
public byte NoiseReductionMode { get; private set; }
public bool TouchEnabled { get; private set; }
public bool AutoPlayStopEnabled { get; private set; }
public bool IsValid { get; private set; }
public string ErrorMessage { get; private set; }
public DateTime Timestamp { get; private set; }
}
```

Noise Reduction Mode

```
bool success = await _sdkApi.SetNoiseReductionModeAsync(0);
```

Set whether to enable noise reduction or ambient sound mode.

Parameter values:

- **0** — Normal mode
- **1** — Noise reduction mode
- **2** — Ambient sound

Touch Control

```
bool success = await _sdkApi.SetTouchEnabledAsync(true);
```

Set whether touch control is enabled.

Parameter values:

- **true** — On
- **false** — Off

Smart Playback

```
bool success = await _sdkApi.SetAutoPlayEnabledAsync(true);
```


Set whether smart playback is enabled.

Parameter values:

- `true` — On
- `false` — Off

Start Data Acquisition

```
bool success = await _sdkApi.SendStartDataAsync();
```

Start data acquisition. After starting, the `SdkApi_DataReceived` callback will periodically receive data.

Stop Data Acquisition

```
bool success = await _sdkApi.SendStopDataAsync();
```

Stop data acquisition. After stopping, `SdkApi_DataReceived` will no longer receive data.

Raw EEG Data

```
private void SdkApi_DataReceived(object sender, DataReceivedEventArgs e)
```

After connection and starting data acquisition, the earbuds periodically send left/right ear EEG data to the host via BLE.

Structure:

```
public class DataReceivedEventArgs : EventArgs
{
    public EarSide EarSide { get; private set; } // 0: Left ear, 1: Right ear
    public float[] Data { get; private set; } // Raw data
    public byte PacketCounter { get; private set; } // Packet counter for detecting
```

```

packet loss
    public bool IsValid { get; private set; }      // Whether data is valid
    public string ErrorMessage { get; private set; } // Error message
    public DateTime Timestamp { get; private set; } // Timestamp
    public byte[] RawData { get; private set; }    // 24-bit little-endian raw data
}

```

Get Latest 1–60 Seconds of EEG Data

```

float[] leftData = _sdkApi.GetLatestEegData(EarSide.Left, _displaySeconds,
_useFilteredData);
float[] rightData = _sdkApi.GetLatestEegData(EarSide.Right, _displaySeconds,
_useFilteredData);

```

The SDK stores up to 60 seconds of left/right ear raw data and filtered data.

Parameter description:

- **EarSide**: 0 for left ear, 1 for right ear
- **_displaySeconds**: 0–60 seconds
- **_useFilteredData**: Whether to use filtered data; **true** = filtered, **false** = raw data

Signal Quality & Spectrum Data

```

private void SdkApi_SpectrumDataReceived(object sender, SpectrumDataEventArgs e)

```

After starting data acquisition, this callback reports current signal quality, delta, theta, alpha, beta, gamma, etc. once per second.

Enum:

```

public enum SignalQuality
{
    OK,
    SizeError,
    OriginalMaxValue,
    OriginalZeroValue,
    FilteredMaxValue,
    FilteredZeroValue,
    ThresholdError,
}

```

```
PowerFrequencyError,  
OtherError  
}
```

Structure:

```
public class SpectrumDataEventArgs : EventArgs  
{  
    public double[] LeftDelta { get; private set; }  
    public double[] LeftTheta { get; private set; }  
    public double[] LeftAlpha { get; private set; }  
    public double[] LeftBeta { get; private set; }  
    public double[] LeftGamma { get; private set; }  
    public double[] LeftLowAlpha { get; private set; }  
    public double[] LeftHighAlpha { get; private set; }  
    public double[] LeftLowBeta { get; private set; }  
    public double[] LeftHighBeta { get; private set; }  
    public double[] LeftLowGamma { get; private set; }  
    public double[] LeftHighGamma { get; private set; }  
    public double[] RightDelta { get; private set; }  
    public double[] RightTheta { get; private set; }  
    public double[] RightAlpha { get; private set; }  
    public double[] RightBeta { get; private set; }  
    public double[] RightGamma { get; private set; }  
    public double[] RightLowAlpha { get; private set; }  
    public double[] RightHighAlpha { get; private set; }  
    public double[] RightLowBeta { get; private set; }  
    public double[] RightHighBeta { get; private set; }  
    public double[] RightLowGamma { get; private set; }  
    public double[] RightHighGamma { get; private set; }  
    public SignalQuality LeftSignalQuality { get; private set; }  
    public SignalQuality RightSignalQuality { get; private set; }  
    public DateTime Timestamp { get; private set; }  
}
```

Real-Time Mental State Feedback

```
bool success = _sdkApi.StartMentalStateTask(1);
```

Parameter: Interval for periodic feedback, 0.5–5 seconds.

Structure:

```
public class MentalStateDataEventArgs : EventArgs  
{  
    public double Focus { get; private set; }    // Focus level
```

```
public double Fatigue { get; private set; } // Fatigue level
public double Relax { get; private set; } // Relaxation level
public double Calm { get; private set; } // Calmness level
public double Stress { get; private set; } // Stress level
public DateTime Timestamp { get; private set; }
}
```

Save CSV Example

After recording ends, data is automatically saved to

`data\YYYY\MM\EEG_Data_YYYYMMDD_hhmmss.xlsx`.

The sample includes a CSV format save example:

- Left and right ear data are recorded separately, with BLE receive timestamps and 50 float values per packet, separated by commas.
- Start recording: `BtnStartRecording_Click`
- Stop recording: `BtnStopRecording_Click`
- Save: `SaveDataToCsv` — the example uses the open-source EPPlus library (version 4.5.3.3) for Excel file storage.

Real-Time Waveform Example

The sample includes a real-time waveform refresh example:

- Uses the open-source `ScottPlot.WinForms FormsPlot` control.
- Raw data is obtained via `_sdkApi.GetLatestEegData`.
- Data refresh uses a `Timer` with a 100ms update interval.

Real-Time Spectrum Example

The sample includes a real-time spectrum example:

- Uses the open-source `ScottPlot.WinForms FormsPlot` control.

- Data is updated every second via the `_sdkApi.SpectrumDataReceived` callback.